

Concurrencia

Concurrencia

Un programa se dice **concurrente** si **distintas partes del programa pueden ejecutarse en simultáneo**.

Podemos distinguir cuatro niveles en los cuales puede haber concurrencia::

1. **A nivel de instrucción**, donde dos o más instrucciones máquina se ejecutan en simultáneo.
2. **A nivel de sentencia**, donde dos sentencias del lenguaje se ejecutan en simultáneo.
3. **A nivel de unidad**, donde dos o más unidades del programa se ejecutan en simultáneo.
4. **A nivel de programa**, donde dos o más programas se ejecutan en simultáneo.



Algunos **beneficios** de la programación concurrente consisten en un mejor modelado de ciertos problemas y escenarios, un mejor rendimiento de los programas, y la posibilidad de ejecución de un sistema de manera distribuida.

Concurrencia

El concepto fundamental para concurrencia a nivel de unidad es el de tarea.

Una tarea es una unidad del programa que puede ejecutarse de forma concurrente con otras unidades del mismo programa.

Las tareas tienen algunas características que las distinguen de las unidades tradicionales:

- Su ejecución puede comenzar de forma implícita.
- Cuando una tarea es invocada, no necesariamente se suspende la ejecución de la unidad llamadora.
- Cuando la ejecución de la tarea es completada, el control puede no retornar a la unidad llamadora.

Para permitir programación concurrente, un lenguaje de programación debe brindar **mecanismos para resolver la comunicación y la sincronización** entre tareas.

Comunicación

El lenguaje de programación debe brindar algún mecanismo que posibilite que una tarea consiga información producida por otra tareas.

Los mecanismos más frecuentes son los modelos de **memoria compartida** y **pasaje de mensajes**.

En el **modelo de memoria compartida**, algunas (o todas) de las variables del programa son accesibles para varias tareas. Para comunicarse, una tarea escribe un valor en una variable y otra tarea lo lee.

En el **modelo de pasaje de mensajes**, las tareas no tienen un estado común. Para comunicarse, una tarea debe hacer explícitamente un envío de información a otra tarea.

Sincronización

La sincronización permite controlar el orden relativo de las operaciones pertenecientes a diferentes tareas. En muchos casos, la sincronización resulta fundamental para lograr la correctitud del programa.

Podemos distinguir dos tipos de sincronización:

- **Sincronización cooperativa**, cuando las tareas cooperan para alcanzar algún objetivo. Una tarea deberá esperar que otra tarea finalice una actividad específica antes de poder continuar con su ejecución.
- **Sincronización competitiva**, cuando las tareas deben usar un mismo recurso, el cual no puede ser utilizado simultáneamente.

Los tres mecanismos típicos para sincronizar tareas son los **semáforos**, **monitores** y **pasaje de mensajes**.

Sincronización

Un **semáforo** es una estructura de datos que consiste de un contador y una cola que almacena descriptores de tareas.

Sincronización cooperativa y competitiva queda a cargo del programador a través de las operaciones del semáforo.

Es un mecanismo propenso a errores y el semáforo es, en sí mismo, un recurso compartido por las tareas.

```
semaforo S = nuevo Semaforo(1)
recursoCompartido = 0

proceso Tarea1:
    S.wait()
    imprimir("Tarea1 accede al recurso")
    recursoCompartido = recursoCompartido + 1
    dormir(1 segundo)
    imprimir("Tarea1 actualiza recurso a", recursoCompartido)
    S.release()

proceso Tarea2:
    S.wait()
    imprimir("Tarea2 accede al recurso")
    recursoCompartido = recursoCompartido * 10
    imprimir("Tarea2 actualiza recurso a", recursoCompartido)
    S.release()

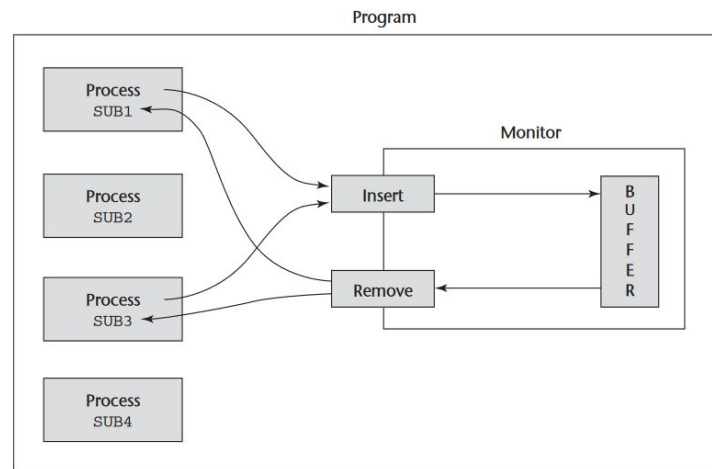
iniciar Tarea1
iniciar Tarea2
```

Sincronización

Un **monitor** es una abstracción que encapsula un recurso compartido y brinda un conjunto de operaciones para operar con ese recurso.

Provee de manera natural sincronización competitiva, donde el recurso compartido y la gestión de las tareas que lo requieren queda a cargo del monitor, garantizando que solo una tarea a la vez pueda acceder a las operaciones críticas.

La sincronización cooperativa sigue siendo responsabilidad de las tareas.



Sincronización

El mecanismo de **pasaje de mensajes** permite que una tarea envíe un aviso a otra con la cual desea comunicarse o sincronizarse.

Si la tarea receptora está disponible, la comunicación tiene lugar, mientras que si está ocupada, la tarea emisora queda en espera.

```
task body Machine_Expendedora is
  Sodas_En_Stock  : Integer := 5; -- Estado interno
  Snacks_En_Stock : Integer := 3; -- Estado interno
begin
  loop
    select
      when Sodas_En_Stock > 0 =>
        accept Dispense_Soda (Recibido : out Boolean) do
          -- Lógica de dispensar soda, actualiza Sodas_En_Stock
          end Dispense_Soda;
      or
        when Snacks_En_Stock > 0 =>
          accept Dispense_Snack (Recibido : out Boolean) do
            -- Lógica de dispensar snack, actualiza Snacks_En_Stock
            end Dispense_Snack;
    end select;
  end loop;
end Machine_Expendedora;
```


Concurrencia en Java

Java utiliza hilos para expresar concurrencia, a través de la clase Thread. Las unidades concurrentes serán métodos run(), provisto por la clase Thread.

Una clase que quiera tener un método concurrente debe heredar de la clase Thread e implementar el método run().

```
class EjemploThread extends Thread {  
    private String name;  
  
    public EjemploThread(String name) {  
        this.name = name;  
    }  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Hola desde " + name + "  
                - Contador: " + i);  
        }  
        System.out.println(name + " finalizado.");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
  
        // Crear dos hilos  
        Thread thread1 = new EjemploThread("Hilo 1");  
        Thread thread2 = new EjemploThread("Hilo 2");  
  
        // Iniciar los hilos  
        thread1.start();  
        thread2.start();  
    }  
}
```

Concurrencia en Java

Podemos ejemplificar las facilidades que brinda Java a través del **problema de Productor - Consumidor**.

- Un Productor se encarga de guardar un dato en el Depósito.
- Un Consumidor se encarga de leer un dato del Depósito.
- El productor no puede guardar un nuevo dato si el Depósito ya tiene uno.
- El consumidor no puede leer un dato si el Depósito no tiene uno.



Productor y Consumidor serán procesos concurrentes que extienden a la clase Thread.

Concurrencia en Java

Podemos ejemplificar las facilidades que brinda Java a través del **problema de Productor - Consumidor**.

- Un Productor se encarga de guardar un dato en el Depósito.
- Un Consumidor se encarga de leer un dato del Depósito.
- El productor no puede guardar un nuevo dato si el Depósito ya tiene uno.
- El consumidor no puede leer un dato si el Depósito no tiene uno.



Productor y Consumidor serán procesos concurrentes que extienden a la clase Thread.

Concurrencia en Java

Podemos ejemplificar las facilidades que brinda Java a través del **problema de Productor - Consumidor**.

- Un Productor se encarga de guardar un dato en el Depósito.
- Un Consumidor se encarga de leer un dato del Depósito.
- El productor no puede guardar un nuevo dato si el Depósito ya tiene uno.
- El consumidor no puede leer un dato si el Depósito no tiene uno.



Productor y Consumidor serán procesos concurrentes que extienden a la clase Thread.

Concurrencia en Java

Productor y Consumidor deben estar sincronizados para acceder de manera exclusiva al depósito.

```
class Productor extends Thread {  
  
    Depósito D;  
    Productor (Depósito dep){  
        D = dep;  
    }  
  
    void Run(){  
        while(true){  
            D.put(new Item());  
        }  
    }  
}
```

```
class Consumidor extends Thread {  
  
    Depósito D;  
    Consumidor (Depósito dep){  
        D = dep;  
    }  
  
    void Run(){  
        while(true){  
            item i = D.get();  
        }  
    }  
}
```

Concurrencia en Java

En la clase Depósito, la variable *data* contiene el valor a producir y leer, y la variable *withData* indica si el depósito tiene o no un valor.

En el método *get* se devuelve el valor del depósito. Si no hay un valor aún, se suspende al consumidor hasta que un productor guarde un nuevo valor.

```
public class Depósito {
    private int data;
    private boolean withData = false;

    public synchronized int get() {
        try {
            while (!withData) {
                wait();
            }
            withData = false;
            notifyAll();
        } catch (InterruptedException e) { ... }
        return data;
    }

    public synchronized int put(int value) {
        ...
    }
}
```

Concurrencia en Java

En la clase `Depósito`, la variable *data* contiene el valor a producir y leer, y la variable *withData* indica si el depósito tiene o no un valor.

En el método *get* se devuelve el valor del depósito. Si no hay un valor aún, se suspende al consumidor hasta que un productor guarde un nuevo valor.

En el método *put* se almacena un nuevo dato. Si todavía hay un dato en el depósito, se suspende al productor hasta que ese valor sea leído.

```
public class Depósito {
    private int data;
    private boolean withData = false;

    public synchronized int get() {
        try {
            while (!withData) {
                wait();
            }
            withData = false;
            notifyAll();
        } catch (InterruptedException e) { ... }
        return data;
    }

    public synchronized int put(int value) {
        try {
            while (withData) {
                wait();
            }
            data = value;
            withData = true;
            notifyAll();
        } catch (InterruptedException e) { ... }
        return data;
    }
}
```

Caso de Estudio - Go

Introducción

Go es un lenguaje del paradigma imperativo de propósito general, con tipado estático, y una sintaxis inspirada en C.

Se destacan las facilidades que brinda para realizar programación concurrente de una manera sencilla y clara.

- Goroutines
- Channels
- select
- WaitGroups
- Mutex y RWMutex

[Compilador online de Go.](#)

```
package main

import (
    "fmt"
    "time"
)

func emisor(ch chan string) {
    time.Sleep(1 * time.Second)
    ch <- "¡Hola desde la goroutine!"
}

func main() {

    mensajes := make(chan string)

    go emisor(mensajes)

    mensajeRecibido := <-mensajes

    fmt.Println(mensajeRecibido)
    fmt.Println("Programa finalizado.")
}
```

Variables y Constantes

Go utiliza **tipado estático**, con lo cual el tipo de una variable no puede cambiar una vez que haya sido ligado.

El lenguaje brinda tres maneras de declarar variables:

- Utilizando **var**, indicando el tipo de la variable.
- Utilizando **var**, sin indicar el tipo de la variable.
- De forma abreviada, asignándole un valor a la variable.

Para variables sin valor, Go asigna un valor por defecto dependiendo del tipo de la variable.

Una **constante** se declara usando **const** seguida del nombre de la constante, el tipo de dato y el valor asignado. **El valor debe poder ser determinado en tiempo de compilación.**

```
func main() {  
    var a int  
    a = 10  
    fmt.Println(a)           // 10  
  
    var b = 15  
    fmt.Println(b)           // 15  
  
    var c, d int = 1, 2  
    fmt.Println(c, d)        // 1 2  
  
    e := 5  
    fmt.Println(e)           // 5  
  
    var z int  
    fmt.Println(z)           // 0  
  
    const pi float64 = 3.142  
    fmt.Println(pi)          // 3.142  
}
```

Tipos Básicos

En Go, los tipos básicos se dividen principalmente en tres grandes categorías: numéricos, booleanos y cadenas de texto.

Numéricos

- int int8 int16 int32 int64, ...
- rune, byte
- float32 float64
- complex64 complex128

Booleanos

- bool

Cadenas

- string

```
func main() {  
    edad := 30           // Numérico entero  
    peso := 75.5         // Numérico de punto flotante  
    esValido := true     // Booleano  
    lenguaje := "Go"     // Cadena de texto  
  
    // Impresión de valores y tipos  
    fmt.Printf("Var edad : %v | Tipo: %T\n", edad, edad)  
    fmt.Printf("Var peso : %v | Tipo: %T\n", peso, peso)  
    fmt.Printf("Var esValido : %v | Tipo: %T\n", esValido, esValido)  
    fmt.Printf("Var lenguaje : %v | Tipo: %T\n", lenguaje, lenguaje)  
}
```

Tipos Básicos

Además, se cuenta con punteros.

Dos operadores:

- `[]` para acceder o modificar el valor guardado en la dirección de memoria correspondiente.
- `&` para obtener la ubicación exacta en la memoria de una variable.

Go no permite la aritmética de punteros.

```
func main() {  
    i, j := 42, 2701  
    var p *int  
  
    p = &i           // apuntar a i  
    fmt.Println(*p) // leer i a través del puntero  
    *p = 21          // modificar i mediante el puntero  
    fmt.Println(i)  // ver el nuevo valor de i  
  
    p = &j           // apuntar a j  
    *p = *p / 37     // dividir j a través del puntero  
    fmt.Println(j)  // ver el nuevo valor de j  
}
```

Tipos Estructurados

Como tipos estructurados, se destacan los **arrays**, **slices**, **structs** y **maps**.

Un **array** es una secuencia de elementos del mismo tipo, y de un tamaño fijo.

Los **slices** dan mayor flexibilidad al programador ya que su tamaño es dinámico.

La forma usual de crear un *slice* o un *map* es usando la función *make*.

```
func main() {  
    // Arrays  
    var a [2]int  
    fmt.Println("array :", a) // array : [0 0]  
  
    a[0] = 1  
    a[1] = 2  
    fmt.Println("array :", a) // array : [1 2]  
  
    b := [2]int{3, 4}  
    fmt.Println("array :", b) // array : [3 4]  
  
    // Slices  
    s := make([]int, 3)  
    fmt.Println("slice:", s) // slice: [0 0 0]  
    fmt.Println("length:", len(s)) // length: 3  
    fmt.Println("capacity:", cap(s)) // capacity: 3  
  
    s[1] = 1  
    fmt.Println("slice:", s) // slice: [0 1 0]  
    s = append(s, 1)  
    fmt.Println("slice:", s) // slice: [0 1 0 1]  
}
```

Tipos Estructurados

Un **struct** es un tipo definido por el usuario que permite agrupar campos de distintos tipos.

Un **map** permite definir una colección de pares clave-valor. Para declarar un map usaremos la función *make*. Al definirlo, debemos indicar el tipo para las claves y para los valores.

```
type person struct {  
    name string  
    age  int  
}  
func main() {  
    // Structs  
    fmt.Println(person{"Federico", 29}) // {Federico 29}  
  
    unaPersona := person{name: "Federico", age: 29}  
    fmt.Println(unaPersona.name) // Federico  
    fmt.Println(unaPersona.lastName) // error  
  
    fmt.Println(person{name: "Martín"}) // {Martín 0}  
}
```

```
func main() {  
    // Maps  
    m := make(map[string] int)  
    m["k1"] = 7  
    m["k2"] = 13  
    fmt.Println("map:", m) // map: map[k1:7 k2:13]  
  
    delete(m, "k2")  
    fmt.Println("map:", m) // map: map[k1:7]  
  
    v1, ok := m["k1"]  
    fmt.Println("v1:", v1, "ok:", ok) // v1: 7 ok: true  
}
```

Control de Flujo :: Asignación

Las asignaciones en Go permiten asignar un valor a una variable. Se puede realizar de dos formas:

- *variable = valor*, en caso de que *variable* haya sido declarada previamente.
- *variable := valor*, en cuyo caso *variable* se declara e inicializa en la misma sentencia.

```
func main() {  
    var a int  
    a = 10  
    fmt.Println(a)    // 10  
  
    e := 5  
    fmt.Println(e)    // 5  
  
    const pi float64 = 3.14259  
    fmt.Println(pi)   // 3.14259  
}
```

Control de Flujo :: Selección

Para selección, el lenguaje brinda una sentencia *if-else* y una sentencia *switch*.

La sentencia *if-else* nos permite tomar una decisión en base a una condición.

Es posible incluir una sentencia antes de la expresión condicional. Cualquier variable declarada antes de la condición estará disponible en toda la sentencia condicional.

Cuenta con los operadores booleanos tradicionales: `<`, `>`, `<=`, `>=`, `==`, `!=`, `&&` y `||` (estos últimos en cortocircuito).

```
func main() {  
    n := 1  
    if n > 0 {  
        fmt.Println("N es positivo")  
    }  
  
    if num := 9; num < 0 {  
        fmt.Println(num, "es negativo")  
    } else if num < 10 {  
        fmt.Println(num, "tiene un dígito")  
    } else {  
        fmt.Println(num, "tiene varios dígitos")  
    }  
  
    fmt.Println(num) // error  
}
```


Control de Flujo :: Selección

Para selección, el lenguaje brinda una sentencia *if-else* y una sentencia *switch*.

La sentencia *switch* permite ejecutar distintos bloques en base al valor de una expresión.

Algunas consideraciones:

- Los casos son mutuamente excluyentes, un valor no puede aparecer en más de un caso.
- Como máximo se ejecutará un único caso, salvo que indiquemos lo contrario.
- Puede agregarse un caso “default”.
- “Break” implícito al final de cada caso, pero podemos evitarlo usando la palabra “fallthrough” al final de un caso.

```
func main() {  
    n := 3  
    switch n {  
    case 1:  
        fmt.Println("n es 1")  
    case 2, 3, 4, 5:  
        fmt.Println("n es 2, 3, 4 o 5")  
    }  
  
    switch i := 2 + n; i {  
    case 1, 2, 3, 4, 5:  
        fmt.Println("i está entre 1 y 5")  
    default:  
        fmt.Println("i es mayor a 5")  
    }  
  
    j := 1 + n  
    switch {  
    case j <= 5:  
        fmt.Println("j es menor o igual a 5")  
    case j <= 10:  
        fmt.Println("j entre 5 y 10")  
    default:  
        fmt.Println("j mayor a 10")  
    }  
}
```

Control de Flujo :: Iteración

Toda iteración se realiza a través de un constructor *for*.

Podemos incluir hasta tres elementos al comienzo del *for*:

- Inicializador de la variable de control.
- Condición de corte.
- Incremento de la variable de control.

Cuenta con instrucciones de *break* y *continue*.

Además, podremos combinar el *for* con el constructor *range* para iterar sobre una colección de elementos (arrays, slices, maps).

```
func main() {  
    for i := 1; i < 5; i++ {  
        fmt.Println(i)  
    }  
}
```

```
func main() {  
  
    j := 1  
    for j < 5 {  
        fmt.Println(j)  
        j = j + 1  
    }  
  
    h := 1  
    for {  
        if h == 5 {  
            break  
        }  
        fmt.Println(h)  
        h = h + 1  
    }  
  
    s := []int{1, 2, 3, 4, 5}  
    for k, v := range s {  
        fmt.Println(k, v)  
    }  
}
```

Control de Flujo :: Defer

La instrucción **defer** permite posponer la ejecución de una función hasta un momento antes de que la función llamadora (que contiene la llamada con defer) haya finalizado.

En una función puede haber más de una llamada diferida, las cuales quedarán pendientes y ordenadas formando una pila (es decir, la última llamada diferida será la primera en ejecutarse).

```
func main() {  
    fmt.Println("Inicio de la función")  
    defer fmt.Println("Fin de la función")  
  
    for i := 0; i < 5; i++ {  
        fmt.Println(i)  
    }  
}
```

Es una instrucción que resulta útil para realizar tareas de terminación, como por ejemplo cerrar un archivo, cerrar un *channel*, disminuir el contador de un *waitGroup*.

Funciones

Definimos una función con la palabra reservada **func**, seguido del nombre de la función y sus parámetros (en caso de tenerlos) entre paréntesis.

Debe indicarse el tipo de cada parámetro y el tipo de cada valor resultado.

También se pueden definir **funciones anónimas**, que en general son invocadas en el mismo lugar en que se definen.

```
func main() {  
    func() {  
        fmt.Println("Hola")  
    } () // función anónima e invocación  
}
```

```
func saludar(msg string) {  
    fmt.Println(msg)  
}  
  
func sumar(a, b, c int) int {  
    return a + b + c  
}  
  
func valores() (int, string) {  
    return 3, "cadena"  
}  
  
func main() {  
    saludar("Hola")  
  
    suma := sumar(1, 2, 3)  
    fmt.Println(suma) // 6  
  
    x, y := valores()  
    fmt.Println(x, y) // 3 cadena  
}
```

Pasaje de Parámetros

En Go, el pasaje de parámetros es por valor.

- Para tipos básicos, arrays, structs y punteros, se crea una copia del valor pasado como argumento al parámetro formal de la función invocada.

```
func foo(b [4]int) int {  
    b[0] = 0  
    return b[0]  
}  
  
func main() {  
    a := [3]int{1, 2, 3, 4}  
    v := foo(a)  
    fmt.Println(v) // 2  
    fmt.Println(a) // [1 2 3 4]  
}
```

Pasaje de Parámetros

En Go, el pasaje de parámetros es por valor.

- Para tipos básicos, arrays, structs y punteros, se crea una copia del valor pasado como argumento al parámetro formal de la función invocada.
- Para strings, slices, maps o channels, se crea una copia del descriptor que contiene punteros a los datos en memoria.

```
type Persona struct {  
    Nombre string  
    Edad   int  
}  
  
func cumplirAños(p *Persona) {  
    p.Edad++  
}  
  
func main() {  
    usuario := Persona{Nombre: "Juan", Edad: 25}  
    fmt.Println("Antes:", usuario.Edad) // 25  
    cumplirAños(&usuario)  
    fmt.Println("Después:", usuario.Edad) // 26  
}
```

```
type Persona struct {  
    Nombre string  
    Edad   int  
}  
  
func reasignando(p *Persona) {  
    p = &Persona{Nombre: "Copia Local", Edad: 99}  
}  
  
func main() {  
    usuario := &Persona{Nombre: "Juan", Edad: 25}  
    fmt.Println("Antes:", usuario.Edad) // 25  
    reasignando(usuario)  
    fmt.Println("Después:", usuario.Edad) // 25  
}
```

Concurrencia en Go :: Goroutines

A diferencia de otros lenguajes de programación que utilizan hilos del sistema operativo, Go permite ejecutar tareas de forma concurrente utilizando **goroutines**.

Una goroutine es una abstracción de un hilo del sistema operativo.

Un planificador en el runtime de Go se encargará de asignar goroutines a hilos del sistema operativo para que sean ejecutadas.

Las goroutines tienen dos grandes ventajas:

- Es menos costoso generarlas y mantenerlas, en comparación con usar hilos del sistema operativo.
- Al programar, no interactuamos con hilos de bajo nivel, sino con abstracciones.

```
func saludar() {  
    fmt.Println("hola")  
}  
  
func main() {  
    go saludar()  
    time.Sleep(100 + time.Millisecond)  
}
```

Concurrencia en Go :: Goroutines

En el caso del problema del Productor - Consumidor, si consideramos únicamente un productor y un consumidor, podemos tener algo así:

```
func productor(ch chan<- string) {  
    // código del productor  
}  
  
func consumidor(ch <-chan string) {  
    // código del consumidor  
}  
  
func main() {  
    canalContenido := make(chan string)  
  
    go productor(canalContenido)  
    go consumidor(canalContenido)  
}
```

Desde main se crean dos goroutines, una para el productor y otra para el consumidor.

¿Qué problemas aparecen en esta implementación?

Concurrencia en Go :: WaitGroup

WaitGroup es una estructura de sincronización que permite esperar a que un grupo de goroutines completen su ejecución antes de continuar con el flujo principal del programa.

```
func saludar() {  
    fmt.Println("hola")  
}  
  
func main() {  
    go saludar()  
    time.Sleep(100 + time.Millisecond)  
}
```



```
var wg = sync.WaitGroup{}  
  
func saludar() {  
    fmt.Println("hola")  
    wg.Done()  
}  
  
func main() {  
    wg.Add(1) // contador de goroutines en espera  
    go saludar()  
    wg.Wait()  
}
```

El flujo principal del programa se bloquea hasta que el contador del WaitGroup llegue a cero. **Las goroutines tienen la responsabilidad de decrementar el contador cuando terminan su trabajo.**

Concurrencia en Go :: Mutex

Un *Mutex* permite que solo una goroutine acceda a una sección crítica del código a la vez. Se usa para evitar condiciones de carrera cuando varias goroutines acceden a datos compartidos.

```
var contador int
var mutex sync.Mutex

func incrementar(wg *sync.WaitGroup) {
    mutex.Lock()
    contador++
    mutex.Unlock()
    wg.Done()
}

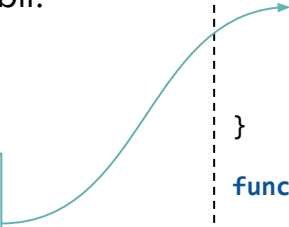
func main() {
    var wg sync.WaitGroup
    for i := 0; i < 1000; i++ {
        wg.Add(1)
        go incrementar(&wg)
    }
    wg.Wait()
    fmt.Println("Contador:", contador)
}
```

La variable contador queda protegida por el mutex, que asegura que solo una goroutine pueda modificarla a la vez.

Concurrencia en Go :: RWMutex

Un *RWMutex* permite que múltiples goroutines lean una sección crítica de forma simultánea, pero exige acceso exclusivo cuando alguna necesita escribir.

Habilitan la lectura concurrente



```
var mu sync.RWMutex
var valor = "inicial"

func leer(wg *sync.WaitGroup) {
    defer wg.Done()
    mu.RLock()
    defer mu.RUnlock()

    fmt.Println("Leído:", valor)
}

func escribir(nuevo string, wg *sync.WaitGroup) {
    defer wg.Done()
    mu.Lock()
    defer mu.Unlock()

    valor = nuevo
}

func main() {
    var wg sync.WaitGroup
    wg.Add(3)

    go leer(&wg)
    go escribir("actualizado", &wg)
    go leer(&wg)
    wg.Wait()
}
```

Concurrencia en Go :: Channels

Los canales (*channels*) permiten sincronizar la transmisión de datos entre goroutines.

Las goroutines pueden intercambiar información sin poner en riesgo la integridad de esa información.

Los canales pueden usarse tanto para resolver la comunicación como la sincronización entre goroutines.

```
func main() {
    ch := make(chan string)

    go func() {
        ch <- "Hola" // Enviar valor
        ch <- "mundo" // Enviar otro valor
        close(ch)
    }()

    msg1 := <- ch // msg1 es "Hola"
    msg2, ok := <- ch // msg2 es "mundo" y ok es true
    msg3, ok2 := <- ch // msg3 es "" y ok es false

    fmt.Println(msg1, msg2) // Hola mundo
}
```

Una goroutine puede enviar un dato por un canal

channelName <- dato

Y puede leer un dato de un canal

variable <- channelName

Concurrencia en Go :: Channels

En el ejemplo anterior, el canal solo almacena un string por vez. Sin embargo, un canal puede almacenar más de un elemento si tiene un búfer de tamaño mayor a 1.

Esto puede ser útil, por ejemplo, si las tareas tienen distinto ritmo de trabajo.

El canal puede almacenar 5 strings.

El productor debe cerrar el canal cuando termine de enviar datos.

Usamos for-range para leer los datos del canal.

```
var wg = sync.WaitGroup{}

func main() {
    ch := make(chan string, 5)

    wg.Add(2)

    go func() {
        for i := 0; i < 5; i++ {
            ch <- "mensaje-" + strconv.Itoa(i)
        }
        close(ch)
        wg.Done()
    }()

    go func() {
        for i := range ch {
            fmt.Println(i)
        }
        wg.Done()
    }()

    wg.Wait()
}
```

Concurrencia en Go :: Channels

Operación	Estado del canal	Resultado
Read	nil	Block
	Abierto y no vacío	Valor
	Abierto y vacío	Block
	Cerrado	<valor default>, false
	Solo escritura	Error en compilación
Write	nil	Block
	Abierto y lleno	Block
	Abierto y no lleno	Escribe valor
	Cerrado	panic
	Solo lectura	Error en compilación
Close	nil	panic
	Abierto y no vacío	Cierra canal y pueden leerse valores
	Abierto y vacío	Cierra canal, lectura produce valor por default
	Cerrado	panic
	Solo lectura	Error en compilación

Concurrencia en Go :: Select

Una goroutine puede esperar y recibir datos de más de un canal.

Para eso, Go brinda una sentencia *select*, que permite a una tarea monitorear varias condiciones (usualmente sobre canales) simultáneamente y actuar cuando alguna de las condiciones se cumple (por ejemplo, se recibe un dato por alguno de los canales).

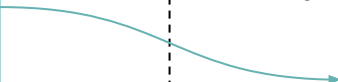
En este caso, la goroutine que está recibiendo datos por los canales es la correspondiente a la función main.

```
func main() {  
    ch1 := make(chan string)  
    ch2 := make(chan int)  
  
    go func() {  
        ch1 <- "Hola"  
    }()  
  
    go func() {  
        ch2 <- 42  
    }()  
  
    select {  
    case msg := <-ch1:  
        fmt.Println("Mensaje recibido:", msg)  
    case num := <-ch2:  
        fmt.Println("Número recibido:", num)  
    case <- time.After(3 * time.Second):  
        fmt.Println("Timeout")  
    }  
}
```

Concurrencia en Go :: Select

En cada momento, el select verifica simultáneamente todos los canales considerados. ¿Qué sucede si, en un momento dado, se verifica más de una condición?

Go selecciona uno de los casos que se cumplen de manera (pseudo) aleatoria



```
func main() {  
    c1 := make(chan interface{})  
    close(c1)  
    c2 := make(chan interface{})  
    close(c2)  
    var c1Count, c2Count int  
  
    for i := 1000; i >= 0; i-- {  
        select {  
            case <-c1: c1Count++  
            case <-c2: c2Count++  
        }  
    }  
  
    fmt.Printf("%d\n %d\n", c1Count, c2Count) // aprox 500 y 500  
}
```


Concurrencia en Go :: Select

En un momento dado, el select puede quedar bloqueado si ninguno de los casos contemplados se cumple (ej. se están monitoreando N canales pero ninguno de ellos tiene un dato para leer). ¿Qué sucede en ese caso?

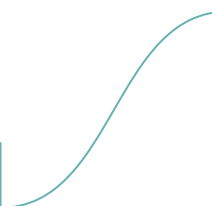
```
func main () {  
    var c <-chan int  
    select {  
    case <-c:  
    case <-time.After(1 * time.Second):  
        fmt.Println("Timed out.")  
    }  
}
```

```
func main() {  
    done := make(chan interface{})  
  
    go func() {  
        time.Sleep(5*time.Second)  
        close(done)  
    }()  
  
    workCounter := 0  
loop:  
    for {  
        select {  
        case <-done: break loop  
        default:  
        }  
  
        workCounter++  
        time.Sleep(1*time.Second)  
    }  
}
```

Concurrencia en Go

En algunos casos, el lenguaje detecta en ejecución que dos o más goroutines están en deadlock, y da un error en ejecución que permite identificar el problema.

fatal error: all goroutines are asleep - deadlock!



```
var mu1, mu2 sync.Mutex

func main() {
    go func() {
        mu1.Lock()
        defer mu1.Unlock()

        // Simula trabajo
        mu2.Lock()
        defer mu2.Unlock()
    }()

    go func() {
        mu2.Lock()
        defer mu2.Unlock()

        // Simula trabajo
        mu1.Lock()
        defer mu1.Unlock()
    }()

    select {}
}
```

¿Preguntas?
